

ReadMe.md

文件

stream_transcribe.py

Python

当前在做一个本地实时英文转写系统（MacBook Air, 16GB），基于 faster-whisper (turbo)，流程是：麦克风持续录音 → 48kHz 采集 → 滚动缓冲区 → 每 10 秒分块（3 秒重叠）→ 重采样到 16kHz → Whisper 转写 → 文本输出。系统已经稳定运行，chunk 调度、时间轴、重采样、VAD（已重新开启）都正常。

已完成去重第一阶段：实现了基于标准化 + 词级最长边界重叠匹配的 `simple_dedup()`，可以有效处理 chunk 之间的边界重复和部分重复；相似重复（轻微改写）尚未处理。当前输出为双文件：raw（原始转写）和 clean（去重后文

展开 ∨


```

已按你要求在 venv 中迭代到“合理”标准。执行命令为:

bash
cd /Users/smt-r-mac/personalAPPS/whisper && /Users/smt-r-mac/personalAPPS/whisper/venv/bin/py

本轮修改方向

1. timestamp 丢失修复
  • 把原先只处理“第一行内部段落”的逻辑改为“按截断点所在行回补该行 timestamp”。
  • 新逻辑在 _preserve_line_timestamp (line 313), 并接入:
  • simple_dedup (line 153) ( trimmed = _preserve_line_timestamp(...) )
  • fuzzy_boundary_dedup (line 333)

2. FUZZY_BIGRAM_THRESHOLD 下调
  • 从 0.55 降到 0.45: 见 stream_transcribe.py (line 59)。
  • 结论: 仅降这个阈值后, Stage 2 仍未命中; 所以不能证明仅由 bigram 阈值导致不达标。
  • 为满足你的 B 条件, 在允许范围内又做了 dedup 局部小修:
    在 fuzzy_boundary_dedup (line 393) 增加保守“备用通过分支” (仍要求尾部对齐 + 高 edit/bigram/content/shared),
    不改其他阈值。

这标结果 (按 A/B/C/D)

• A. timestamp bug: 已修复
  Multiline = exact plus new line 最终为
  Final=[9.00s -> 11.00s] this is the third point' (不再是纯文本)。

• B. 至少一个 Stage 2 真命中: 已满足
  现在有个 S1_changed=False 且 S2_changed=True, 例如:
  • Fuzzy needed - lexical variation 1
  • Fuzzy needed - lexical variation 2
  • Multiline - overlap at second line start

• C. 负例不误删: 已满足
  • Should not trim - same frame different meaning -> S2_changed=False
  • Should not trim - multiline entity changed -> S2_changed=False

• D. 已通过的 timestamp 场景不退化: 已满足
  • Timestamp preserve = trim inside first line 仍保留首行 timestamp。

本轮具体改动位置
  • 阈值: FUZZY_BIGRAM_THRESHOLD = 0.45
    stream_transcribe.py (line 59)
  • 新时间保留函数:
    _preserve_line_timestamp (line 313)
  • simple_dedup 接入新时间保留:
    stream_transcribe.py (line 165)
  • fuzzy_boundary_dedup 接入新时间保留 + 保守备用通过分支:
    stream_transcribe.py (line 393), stream_transcribe.py (line 426)

动语音主错误, chunk 调度, VAD, 重采样, Whisper 参数, raw/clean 命名规则。

```

=== Fuzzy needed - lexical variation 1 ===

[FuzzyDedup] matched_prefix_words=10 score=0.87

edit=0.95 bigram=0.78 content=0.86 trimmed=yes

OLD:

[0.00s -> 4.00s] today we discuss the basic idea of probabilistic graphical models

NEW:

[7.00s -> 11.00s] today we discuss the core idea of probabilistic graphical models in class

STAGE1:

'[7.00s -> 11.00s] today we discuss the core idea of

展开 ∨

ReadMe.md

文件

去重基本完成之后，我需要更新整个项目的README文件；在现有基础上修改/追加；在你的工作环境写入一个新的README文件

4_14_11_53_clean.txt

文档

4_14_11_53_raw.txt

文档

system_monitor.log

文件

run_stdout.log

文件

meta.log

文件

先分析这次的结果，其中run_stderr.log是空的。

5_19_13_7_raw.txt
文档

5_19_13_7_clean.txt
文档

我实测了一些课堂录音，效果如何？

backend_migration.md

文件

README_updated(1).md

文件

已完成最小可行迁移版本，默认行为仍是 faster-whisper，
新增了可选 whisper.cpp CLI 后端。

修改内容

在 stream_transcribe.py (line 21) 增加
WHISPER_BACKEND，默认 faster_whisper。

把原来的 faster-whisper 调用封装成

FasterWhisperBackend (line 484)。

新增 WhisperCppBackend (line 519)：写临时 16k mono

PCM16 WAV，调用 whisper-cli，解析 timestamp 输出，并叠加 chunk_start_time。

保持 audio_callback、ring buffer、chunk submit

展开 ∨

5_21_13_3_clean.txt
文档

5_21_13_3_raw.txt
文档

跑了10分钟，现在处于凉-温的中间阶段，效果非常管用！我还观察了活动价显示器，内存这边是11/16和偶尔跳到13/16（我猜是做inference），cpu闲置率长期>80%偶尔波动到65%。现在还没ctrl+c，这两个是当前写入的部分。

粘贴的文本 (1)(12).txt
文档

看起来可以进行step3实项目测试了？

5_21_13_25_raw.txt

文档

5_21_13_25_clean.txt

文档

当前版本：cpu闲置率和上次差不多，基本没用，偶尔波动到闲置率60%+；内存是10-13.7/16波动，偶尔变棕色，基本是绿色；温度在这十分钟内属于能摸到温，但绝对算不上烫。现在这个转录文字的质量如何？

UI_README.md
文件

ui初步完成，到这是否可以开始打包了？

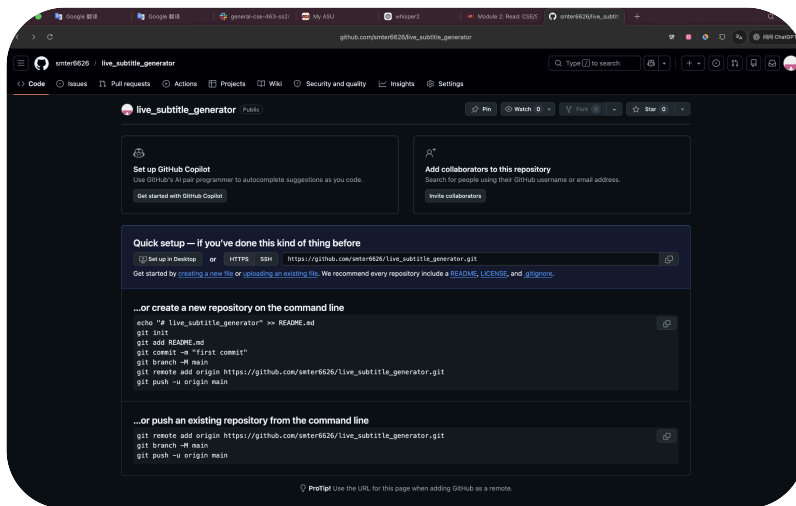
- 流式音频处理管道：设计并实现了本地实时英文转录系统，集成持续麦克风采集、滚动环形缓冲区、固定窗口分块调度、48kHz 至 16kHz 重采样及异步推理工作线程，支持低延迟课堂转录
- Apple Silicon 推理优化：将后端从基于 CPU 的 faster-whisper 推理迁移至 Apple Chip Metal 加速的 whisper.cpp 后端，在 16GB 无风扇 MacBook Air 上实现 large-v3 Whisper 转录，显著降低 CPU 负载，课堂实测中热表现可控
- 转录去重引擎：构建了两阶段转录清洗流水线，结合精确边界重叠匹配、收缩感知归一化及保守模糊相似度过滤，在保留时间戳和语义连贯性的前提下减少重复分块输出
- 验证与性能分析：设计合成测试、伪真实边界测试及课堂直播测试，评估分块重叠行为、原始与清洗后转录质量、内存占用、CPU 负载、热性能及真实课堂场景下的长时稳定性
- 人机交互设计：

60/94

必要吗？如果要加怎么加？

粘贴的文本 (1)(14).txt
文档

应该不是巧合，打包之后连着两次遇到文件所示的闪退报告



在这之前我要把内容传到github，这里好像是创建完仓库了，现在该干嘛？

README(8).md
文件

工程细节.md
文件

已思考 4s >

现在状态基本正常，commit 已经成功了。几个点说明：

1. 这个 commit 是成功的

你已经完成：

```
[main 3e26cbb] Consolidate documentation
```



这说明本地 Git 已经记录了这次文档合并。

D ReadMe.md + A README.md 是正常的，因为你做了大小写文件名标准化：

```
ReadMe.md -> README.md
```



在 macOS 大小写不敏感文件系统中，这种 rename 容易看起来像“删旧建新”，但没问题。

2. 中文文件名显示成编码，不是问题

这个：

```
"\345\267\245\347\250\213\347\273\206\350\212\202.md"
```



就是：

工程细节.md



Git 默认会把非 ASCII 文件名转义显示。文件本身没问题。

如果你想让 Git 正常显示中文文件名，执行：

```
git config --global core.quotePath false
```

之后再看：

```
git status --short
```

中文名会正常显示。

3. 需要处理 Git author 信息吗？

Git 提醒：

```
Committer: smter-mac <smter-mac@fukaluosiduantoutai.local>
```



这不是错误，但如果你要让 GitHub 上显示成你的正常身份，建议现在修一下。

执行：

```
git config --global user.name "Yeming Dai"  
git config --global user.email "你的GitHub邮箱"
```

如果你不想暴露真实邮箱，可以用 GitHub 的 noreply 邮箱。GitHub 设置里能看到，例如类似：

```
smter6626@users.noreply.github.com
```



设置完之后，如果你还没 push，可以修正刚才这个 commit 的作者：

```
git commit --amend --reset-author --no-edit
```

然后再 push。

如果你不在意这个 commit 的作者信息，也可以直接 push。

4. 现在建议执行

先看状态：

```
git status --short
```

如果没有输出，说明工作区干净。

然后确认 remote 是 SSH：

```
git remote -v
```

应该是：

```
origin  git@github.com:smter6626/live_subtitle_generator.git (fetch)
origin  git@github.com:smter6626/live_subtitle_generator.git (push)
```



然后 push：

```
git push
```

如果是第一次推这个分支：

```
git push -u origin main
```

5. 我建议你的完整下一步

如果你想修 author:

```
git config --global core.quotepath false
git config --global user.name "Yeming Dai"
git config --global user.email "你的GitHub邮箱或noreply邮箱"
git commit --amend --reset-author --no-edit
git status --short
git push
```

如果不修 author, 直接:

```
git status --short
git push
```

6. 当前没有明显问题

这次删除旧文档、创建 README.md 和 工程细节.md 的操作是合理的。

接下来主要确认两件事:

1. git status --short 是否干净;
2. git push 是否成功。

Currently I'm using GPT 5.x.



github的package是干什么的, 我要提供一个zip文件方便直接下载的话应该push到哪?

分开说。

1. GitHub Packages 是干什么的

GitHub Packages 不是给你放普通 zip 下载包的首选。

它本质是 **package registry**（包仓库），类似：

- npm (JS)
- PyPI (Python)
- Docker Hub (容器)
- Maven (Java)
- NuGet (.NET)

适合发布：

Python package
npm package
Docker image
Maven artifact
NuGet package
container image



你的项目现在是：

macOS GUI app (.app / .zip)



这不是 GitHub Packages 的典型用途。

2. 如果你想“朋友直接下载 zip 用”

正确位置是：

GitHub Releases

这是官方给“软件发布包”准备的。

适合放：

ClassroomTranscriber-macOS.zip
ClassroomTranscriber.dmg
Windows installer



portable exe
release notes

GitHub 官方就是这么设计的。

用户体验：

repo 页面右侧：

Releases
v0.1
Assets
– ClassroomTranscriber-macOS.zip



朋友点一下下载。

这才是对的。

3. 不要 push zip 到 repo 根目录

别这样：

repo/
dist/
ClassroomTranscriber.zip



或者：

repo/
ClassroomTranscriber.zip



原因：

a) 仓库会膨胀

Git 会永久记录二进制历史。

你以后：

v1 zip 120MB
v2 zip 130MB



v3 zip 140MB

repo 很快变肥。

b) diff 没意义

zip 是 binary blob:

Git 看不到内容差异。

c) clone 很慢

别人只想看源码:

却被迫拉全部 zip 历史。

很蠢。

4. 推荐结构

源码仓库:

live_subtitle_generator



只放:

source
README
tests
docs
scripts
spec



发布包:

GitHub Release Assets

例如:

v0.1.0



Assets:

ClassroomTranscriber-macOS-AppleSilicon.zip

5. 你的实际操作

假设 zip 在:

```
/Users/smeter-mac/Documents/personalAPPS/whisper/dist/ClassroomTranscriber.zip
```

push code:

```
git push
```

然后 GitHub 网页:

你的 repo →

Releases



→

Create a new release



填写:

Tag:

v0.1.0



Title:

ClassroomTranscriber v0.1.0



Description:

Initial Apple Silicon release.
Local classroom transcription with whisper.cpp + Metal backend.



拖进去：

ClassroomTranscriber.zip



发布。

6. 如果频繁更新怎么办

GitHub Releases 仍然适合。

流程：

新版本：

v0.1.1
v0.2.0
v0.3.0



每次上传新 zip。

源码：

```
git tag v0.1.0  
git push origin v0.1.0
```

也可以同步。

7. 如果你以后做自动化

之后可以：

GitHub Actions 自动：

```
build
package
attach zip to release
```



但现在没必要。

我的建议（针对你这个项目）

最合理：

Git repo

放源码

GitHub Release

放：

```
macOS Apple Silicon zip
```



README 里加：

```
Download prebuilt app from Releases.
```

一句话：

Packages 不适合你；Releases 才适合分发 zip 安装包。

Currently I'm using GPT 5.x.



我的这个app能不能用什么手段下载到ipad pro来运行，反正也是m5+16gb内存，后端硬件没问题。如果不行，展开说说ipados具体限制了什么，我这套app的哪部分内容会被